

TP ROS2 Capteurs

19 mai 2026

1 Introduction

Dans ce TP, vous allez apprendre à estimer la trajectoire d'un robot Scout mini + kit de développement ROS2 à partir des données des capteurs. La trajectoire sera reconstruite d'abord en direct, puis avec un enregistrement de données.

Dans une première partie, vous reconstruirez la trajectoire du robot à partir de son odométrie seule, puis à partir de son IMU. Vous observez les limites de ces 2 capteurs en conditions réelles.

Dans une deuxième partie, vous fusionnerez les informations des 2 capteurs précédents pour créer un filtre de Kalman étendu et obtenir une estimation plus fiable de la trajectoire.

Ce TP a pour but de montrer l'intérêt de la fusion des capteurs pour améliorer les estimations de position du robot par exemple.

2 Odométrie

2.1 Définition

L'odométrie est une technique permettant d'estimer la position et l'orientation d'un robot en intégrant ses mouvements au cours du temps.

En robotique mobile, elle repose généralement sur des capteurs internes, comme les encodeurs placés sur les roues, qui mesurent la vitesse de rotation et permettent d'estimer le déplacement du robot. L'odométrie fournit une estimation relative de la position : elle est généralement précise à court terme, mais accumule des erreurs au fil du temps, notamment lors des virages ou en cas de glissement des roues. Elle se distingue des méthodes de localisation absolue (comme le GPS), qui donnent une position globale mais ne reposent pas sur l'intégration du mouvement du robot.

La position du robot est calculée à chaque instant, se basant sur la position initiale connue du robot et en intégrant ses déplacements. Cela revient à intégrer les vitesses linéaires et angulaires du robot. Comme dit plus haut, cette méthode est efficace uniquement à court terme, car intégrer une vitesse implique que s'il y a une erreur, elle est aussi intégrée, ce qui peut entraîner une dérive de la trajectoire du robot.

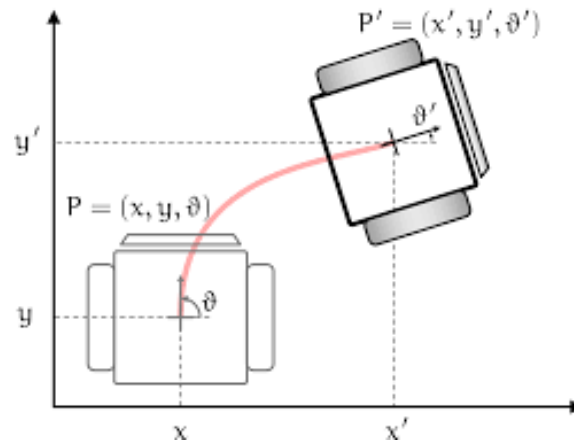


FIGURE 1 – Description odométrie

Dans un cas en 2D, l'estimation de la position du robot à un instant $k+1$ se modéliserai avec le système suivant :

$$\begin{cases} x_{k+1} = x_k + v \cdot \cos(\theta_k) \cdot dt \\ y_{k+1} = y_k + v \cdot \sin(\theta_k) \cdot dt \\ \theta_{k+1} = \theta_k + \omega \cdot dt \end{cases}$$

Avec :

- x_{k+1} la position en x à l'instant $k + 1$
- x_k la position en x à l'instant k
- y_{k+1} la position en y à l'instant $k + 1$
- θ_{k+1} la position angulaire suivant l'axe z à l'instant $k + 1$
- θ_k la position angulaire suivant l'axe z à l'instant k
- v la vitesse linéaire à l'instant k
- dt la dérivée du temps
- ω la vitesse angulaire

2.2 Prise en main du robot et visualisation d'une trajectoire

Pour commencer, si ce n'est pas encore fait, suivez la procédure de mise en marche du robot du TP de mise en marche jusqu'à la partie 4. Connectez vous ensuite au docker avec la commande :

```
docker exec -it gr-p247-humble-container bash
```

Ensuite, visualisez les topics actifs avec la commande :

```
ros2 topic list
```

Identifiez maintenant le topic qui concerne l'odométrie du robot et visualisez les données qu'il publie avec la commande :

```
ros2 topic echo <nom_du_topic>
```

1 : Quelles données sont publiées sur ce topic ?

2 : Comment les données sont-elles composées ? Quel est leur type ? trouvable avec la commande :

```
ros2 topic type <nom_du_topic>
```

- 3 : Que remarquez-vous quand le robot est en déplacement ?
- 4 : Que pouvez-vous en déduire/simplifier ? pas sur de celle la
- 5 : Créez un packet ROS2 avec les commandes :

```
cd src/
ros2 pkg create --build-type ament_cmake <nom_paquet> \
--dependencies rclcpp geometry_msgs sensor_msgs nav_msgs
colcon build
```

Copiez ensuite dans ce nouveau paquet le fichier odom_squelette.cpp, qui contient une node qui s'abonne au topic de l'odométrie et publie une trajectoire sous la forme d'un chemin. Il faut cependant que vous ajoutiez le nom du topic de l'odométrie. Modifiez ensuite le fichier CMakeList en ajoutant les lignes suivantes :

```
add_executable(<nom_node_odometrie> src/<fichier>.cpp)
ament_target_dependencies(<nom_node_odometrie>
  rclcpp
  nav_msgs
  geometry_msgs
)
install(TARGETS
  <nom_node_odometrie>
  DESTINATION lib/${PROJECT_NAME})
```

Dans les prochaines parties, vous devrez refaire cette étape pour chaque nouvelle node que vous créerez.

Maintenant, refaites un colcon build puis lancez la node avec la commande :

```
ros2 run <nom_paquet> <fichier>.cpp
```

Ensuite, allez sur la fenêtre RViz de NoVNC (référez vous au TP de mise en marche du robot). Puis, allez dans le menu displays, faites "Add" -> "by topics" et ajouter le topic que vous avez créé (cf Figure 2). Ensuite, manipuler le robot, vous devriez voir la trajectoire de l'odométrie se tracer directement sur RViz.

6 : Que remarquez-vous sur la trajectoire ? et pendant les virages ? S'il y a trop de points affichés sur RViz, désactivez/décochez le display pointcloud pour enlever le nuage de points générés par le Lidar.

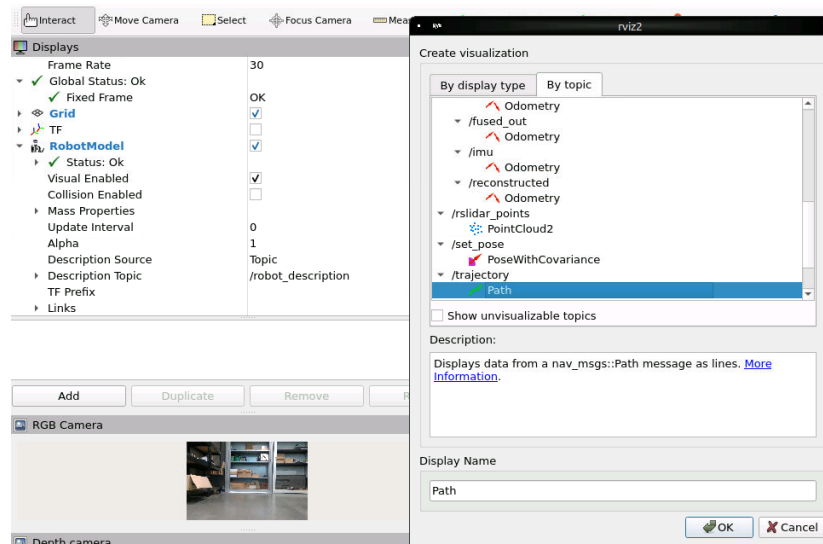


FIGURE 2 – Comment ajouter une trajectoire sur RViz

3 IMU

3.1 Définition

Une Unité de Mesure Inertielle (Inertial Measurement Unit) ou IMU est un composant électronique qui combine des accéléromètres, gyromètres et magnétomètres pour mesurer l'accélération linéaire, la vitesse angulaire d'un robot. Ceci est le contraire de l'odométrie, qui se base uniquement sur le mouvement des roues. Ces données permettent ensuite de calculer l'orientation, c'est à dire les angles d'Euler ϕ (roulis/roll), θ (tangage/pitch) et ψ (lacet/yaw) du robot.

Les angles sont calculés à partir des formules suivantes :

$$\begin{aligned}\phi &= \arctan\left(\frac{a_y}{a_z}\right) \\ \theta &= \arctan\left(\frac{-a_x}{\sqrt{a_y^2 + a_z^2}}\right) \\ \begin{cases} m'_x = m_x \cos \theta + m_z \sin \theta \\ m'_y = m_x \sin \phi \sin \theta + m_y \cos \phi - m_z \sin \phi \cos \theta \\ \psi = \arctan\left(\frac{-m'_y}{m'_x}\right) \end{cases}\end{aligned}$$

Avec :

- a_x, a_y, a_z les composantes de l'accélération linéaire
- m_x, m_y, m_z les composantes du champ magnétique mesuré par le magnétomètre suivant les axes x et y
- m'_x, m'_y les composantes utilisées pour calculer l'angle Yaw compensé (*tilt compensated yaw*)

Cependant, il faut savoir que les angles d'Euler sont pratiques pour comprendre une orientation, mais insuffisants pour un calcul robuste. Cela est du au cas particulier où :

$$\theta = \pm\pi/2$$

Dans ce cas, θ et ψ sont indiscernables, leurs axes se confondent et certaines rotations deviennent impossible à représenter correctement. Ce phénomène s'appelle le *gimbal lock*.

Il faut donc passer par les quaternions.

3.2 Quaternions

Les quaternions sont une manière de représenter une rotation en 3D en évitant le *gimbal lock*. Ils s'écrivent sous la forme :

$$q = \omega + x.i + y.j + z.k$$

où ω représente la partie scalaire du quaternion et x, y et z la partie angulaire.

Une rotation 3D peut être représentée par un axe de rotation $\vec{u}$ et un angle θ autour de cet axe.

Un quaternion peut donc être représenté par :

$$q = \left(\cos\left(\frac{\theta}{2}\right) + \sin\left(\frac{\theta}{2}\right)(u_b i + u_c j + u_d k)\right)$$

soit,

$$q = \left(\cos\left(\frac{\theta}{2}\right), u_b \sin\left(\frac{\theta}{2}\right), u_c \sin\left(\frac{\theta}{2}\right), u_d \sin\left(\frac{\theta}{2}\right)\right)$$

Avec u_b , u_c et u_d des vecteurs unitaires (cf Figure 3).

Les quaternions sont utilisés généralement pour éviter le gimbal lock et car ils sont plus faciles à interpréter et encoder.

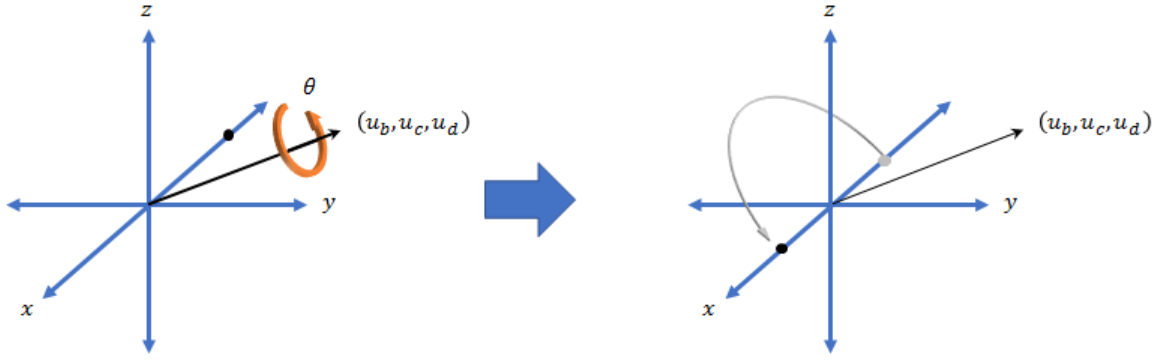


FIGURE 3 – exemple de rotation d'un point

3.3 Des angles d'Euler aux quaternions

Pour rappel, grâce à une IMU, vous pouvez retrouver l'orientation du robot, soit les angles d'Euler. Il faut maintenant convertir ces rotations en quaternions. Pour aller plus vite, voici les quaternions associés à ces rotations :

$$q_{roll} = (\cos(\frac{\phi}{2}), \sin(\frac{\phi}{2}), 0, 0)$$

$$q_{pitch} = (\cos(\frac{\theta}{2}), 0, \sin(\frac{\theta}{2}), 0)$$

$$q_{yaw} = (\cos(\frac{\psi}{2}), 0, 0, \sin(\frac{\psi}{2}))$$

Il nous faut maintenant multiplier ces quaternions afin de n'en avoir qu'un seul représentant l'orientation du robot :

$$q = q_r \otimes q_p \otimes q_y$$

Ce qui nous donne :

$$\begin{cases} \omega = c_r \cdot c_p \cdot c_y + s_r \cdot s_p \cdot s_y \\ x = s_r \cdot c_p \cdot c_y - c_r \cdot s_p \cdot s_y \\ y = c_r \cdot s_p \cdot c_y + s_r \cdot c_p \cdot s_y \\ z = c_r \cdot c_p \cdot s_y - s_r \cdot s_p \cdot c_y \end{cases}$$

Avec :

$$\begin{aligned} - c_r &= \cos(\frac{\phi}{2}), s_r = \sin(\frac{\phi}{2}), \\ - c_p &= \cos(\frac{\theta}{2}), s_p = \sin(\frac{\theta}{2}) \\ - c_y &= \cos(\frac{\psi}{2}), s_y = \sin(\frac{\psi}{2}) \end{aligned}$$

3.4 Visualisation d'une trajectoire

En suivant la même procédure que pour le topic de l'odométrie :

1 : Identifiez les topics liés à l'IMU du robot et leur type.

2 : A partir du fichier imu_traj_squelette.cpp, créez une node qui s'abonne aux topics de l'IMU et publie une trajectoire (double intégration).

3 : Ajoutez la trajectoire sur le display de RViz.

4 : Que remarquez-vous sur la trajectoire ? Est-ce qu'utiliser une IMU pour reconstruire une trajectoire est pertinent ? Pourquoi ?

5 : A partir du fichier imu_orientation_squelette.cpp, créez une node qui publie l'orientation du robot à l'aide des données de l'IMU.

4 Filtre de Kalman

Vous avez vu 2 capteurs séparément (l'odométrie et l'IMU) estimer une trajectoire. Chaque capteur a ses propres limitations, la dérive à long terme pour l'odométrie et les bruits et l'instabilité de l'IMU. Pour améliorer la robustesse de l'estimation de la trajectoire, il devient nécessaire de fusionner les données pertinentes des 2 capteurs. Ceci peut être réalisé par un filtre de Kalman.

4.1 Définition

Un filtre de Kalman est un algorithme permettant d'estimer l'état d'un système à partir de données mesurées. L'algorithme est en 2 étapes : une de prédiction de l'état du système et une qui affine/met à jour l'estimation de l'état du système à partir de mesures bruitées.

En robotique, un filtre de Kalman peut fusionner les données de plusieurs capteurs pour renvoyer la position et l'orientation d'un robot ou générer une carte en corrigeant le glissement des roues par exemple. Cependant, un filtre de Kalman classique est utilisable uniquement si le système suit un modèle mathématique linéaire. Cela n'est pas le cas d'un robot, il faut donc employer un autre type de filtre, un filtre de Kalman étendu ou EKF (Extended Kalman Filter). En bref, ce type de filtre utilise une approximation linéaire du système non linéaire pour réduire la complexité.

4.2 Programmation

Pour cette partie, vous allez utiliser le packet robot-localization de ROS2. Ce packet contient plusieurs node permettant d'estimer l'état d'un robot. Il contient notamment la node ekf, permettant d'implémenter un filtre de Kalman étendu et de fusionner les données de l'IMU et de l'odométrie. La fusion de ces données est ensuite publiée dans un topic appelé */odometry/filtered*.

D'abord, installez le package avec la commande :

```
sudo apt install ros-humble-robot-localization
```

Ensuite, il va falloir créer un fichier permettant de lancer plusieurs node en même temps, un launch file. Pour ce faire, créez un nouveau dossier dans votre paquet et nommez le launch. C'est dans ce dossier que vous créerez votre launch file. Ensuite, ajoutez dans CMakeList :

```
install(DIRECTORY launch
  DESTINATION share/${PROJECT_NAME})
```

1 : A partir du fichier live_traj_squelette.launch.py, créez un fichier lançant toutes les nodes des parties précédentes ainsi que la node ekf de robot-localization et une node publiant la trajectoire du topic */odometry/filtered*.

2 : Visualisez ensuite, sur RViz, les différentes trajectoires publiées. Que remarquez-vous pour la trajectoire représentant la fusion des capteurs ?

Actuellement, il y a un paramètre qui est mis à 0 d'office, il s'agit de la matrice de covariance. Cette matrice indique l'incertitude sur la mesure d'un capteur. Plus la valeur de cette matrice est faible, plus la mesure est précise et inversement. Dans un filtre de Kalman, cette donnée est utilisée pour pondérer les différents capteurs, les mesures plus précises influencent davantage l'estimation finale. Les mesures les plus incertaines ont, quant à elle, une influence moindre.

Donc, notre filtre suit entièrement l'odométrie non filtrée du robot, puisque la covariance est à 0.

3 : Ajoutez une node permettant de modifier la covariance et de republier les données ainsi modifiées de l'odométrie. Pour cela, basez-vous sur le fichier odom_covariance_squelette.cpp et modifiez les paramètres de covariance du fichier où l'orientation du robot est renvoyée. N'oubliez pas d'ajouter la nouvelle node et de changer le nom du topic associé à odom0 dans la node EKF du fichier launch.

4 : Modifiez les valeurs de la matrice de covariance de l'IMU et de l'odométrie et observez les différentes trajectoires. Que remarquez-vous ?

5 Trajectoire pré-enregistrées

Précédemment, vous avez observé la reconstruction d'une trajectoire en direct. Du aux circonstances du TP, vous n'avez sûrement pas pu faire des tests sur de longue trajectoire.

Le but de cette partie est de refaire la partie 4, mais en utilisant une trajectoire pré-enregistrée. Pour ce faire, vérifiez si le packet `ros2 bag` est bien installé ou installez le avec la commande :

```
sudo apt-get install ros-humble-rosbag2
```

Normalement, il faudrait ensuite enregistrer les données d'un ou plusieurs topics sur une durée relativement longue avec la commande :

```
ros2 bag record <topic>
```

Cependant, un enregistrement (ou un bag ici) a déjà été fait dans le dossier XXX. Vous pouvez le lancer avec la commande :

```
ros2 bag play XXX
```

Vérifiez ensuite sur RViz si les trajectoires se tracent. Si ce n'est pas le cas, faites la commande :

```
ros2 topic info /odom --verbose
```

Cette commande devrait vous indiquer en détails les informations du topic `/odom`. Regardez le nombre de publishers, s'il est supérieur à 1, le problème vient de là. Comme plusieurs nodes publient le même topic, les informations du bag entrent en conflit avec celles des autres nodes, empêchant de publier des trajectoires avec des données.

Pour résoudre ce problème, vous avez 2 possibilités : tuer les autres nodes publiant le topic

- tuer les autres nodes publiant le topic, risqué car certaines nodes sont importantes pour le bon fonctionnement du robot ;
- remapper les données vers un nouveau topic, c'est-à-dire créer un nouveau topic au lancement du bag ou de la node.

Partons sur la seconde option pour éviter le plus de risque. Pour ce faire, utilisez cette commande :

```
ros2 bag play XXX --remap /odom:=/odom_bag /imu/data_raw:= /imu/data_raw_bag  
/imu/mag:=/imu/mag_bag
```

Ensuite, modifiez les topics associés aux nodes dans votre fichier launch. Enfin, lancez le fichier puis jouer le bag et vous devriez visualiser les trajectoire enregistrée.